🐍 Wykład 0: Python — od podstaw do pandas i PySpark

Fundamenty programowania dla studentów z różnych backgroundów technicznych

Python w Zastosowaniach Big Data — Studia stacjonarne, Semestr Zimowy 2026/2027

🎯 Agenda

1. Dlaczego Python (i dlaczego właśnie w Big Data)?
2. Zmienne, typy danych, operatory
3. Sterowanie przepływem — `if` / `else`, pętle `for` / `while`
4. Struktury danych — listy, krotki, zbiory, słowniki (+ ich "triki")
5. Funkcje, błędy i wyjątki, odrobina OOP
6. pandas — DataFrame jako "Excel na sterydach"
7. PySpark — gdy dane nie mieszczą się w jednym komputerze

🤔 Zanim zaczniemy...

Ten wykład jest dla **wszystkich** — niezależnie od tego, czy:

- 👩‍💻 programowałeś/aś już w Javie, C# albo JavaScript
- 📊 znasz Excela i SQL, ale nie programowanie
- 🆕 Python to Twój pierwszy język programowania

Cel: jedna wspólna baza, na której zbudujemy całą resztę kursu (Spark, ML, Cloud).

> 💡 Jeśli coś już znasz — świetnie, potraktuj to jako szybkie powtórzenie i szukaj "pythonowych" trików, których może jeszcze nie znasz.

🔥 Dlaczego Python?

- **Czytelna składnia** — kod czyta się niemal jak angielski
- **Ogromny ekosystem** — pandas, NumPy, scikit-learn, PySpark, TensorFlow...
- **Język "klejący"** — łączy bazy danych, API, chmurę, Big Data w jednym miejscu
- **Standard w Data Science i Big Data** — Spark, Airflow, większość narzędzi ML ma Python API

```
# Ten sam problem w Javie to ~10 linii, w Pythonie:
liczby = [1, 2, 3, 4, 5]
suma_kwadratow = sum(x**2 for x in liczby)
print(suma_kwadratow)  # 55
```

🧱 Zmienne i podstawowe typy danych

Python jest **dynamicznie typowany** — nie deklarujesz typu, interpreter sam go rozpoznaje.

```
imie: str = "Ada"            # tekst (string)
wiek: int = 29                # liczba całkowita
wzrost: float = 1.68          # liczba zmiennoprzecinkowa
student: bool = True          # wartość logiczna
nic: None = None              # brak wartości

print(type(wiek))             # <class 'int'>
```

> 💡 Adnotacje typów (`: str`, `: int`) są **opcjonalne**, ale bardzo pomagają w czytelności i są standardem w Python 3.10+.

⚙️ Operatory — szybkie przypomnienie

Kategoria	Operatory	Przykład
Arytmetyczne	`+ - * / // % **`	`7 // 2` → `3`, `7 % 2` → `1`
Porównania	`== != < > <= >=`	`3 == 3.0` → `True`
Logiczne	`and or not`	`True and False` → `False`
Przypisania	`= += -= *= /=`	`x += 1`
Przynależności	`in`, `not in`	`3 in [1, 2, 3]` → `True`

```
cena = 19.99
cena_po_rabacie = cena * 0.9   # += *= itd. działają też tutaj
print(f"{cena_po_rabacie:.2f} zł")   # f-string + formatowanie
```

🧵 Stringi — więcej niż tekst

```
imie = "ada lovelace"

imie.title()                # 'Ada Lovelace'
imie.upper()                 # 'ADA LOVELACE'
imie.split()                  # ['ada', 'lovelace']
"-".join(imie.split())        # 'ada-lovelace'
imie.replace("ada", "grace")  # 'grace lovelace'
imie[0]                       # 'a' (indeksowanie)
imie[::-1]                    # 'ecalevol ada' (slicing z krokiem -1)

f"Cześć, {imie.title()}! Masz {2026 - 1815} 'lat' od urodzenia 😄"
```

> 🎯 **Think about it** 🤔: stringi są *niemutowalne* (immutable) — każda operacja tworzy NOWY string, nie modyfikuje starego!

🔀 Instrukcje warunkowe `if` / `elif` / `else`

```
ocena = 4.5

if ocena >= 5.0:
    print("Celujący! 🏆")
elif ocena >= 4.0:
    print("Bardzo dobry 👍")
elif ocena >= 3.0:
    print("Zaliczone ✅")
else:
    print("Spróbuj jeszcze raz 💪")
```

Python 3.10+: `match` / `case` (odpowiednik `switch`)

```
match ocena:
    case 5.0:
        print("Celujący!")
    case x if x >= 3.0:
        print("Zaliczone")
```

🔁 Pętla `for` — iteracja po sekwencjach

```
owoce = ["jabłko", "banan", "gruszka"]

for owoc in owoce:
    print(f"Kupuję: {owoc}")

# range() – klasyczna pętla "po indeksach"
for i in range(5):              # 0, 1, 2, 3, 4
    print(i)

# enumerate() – indeks + wartość naraz (pythonic way!)
for i, owoc in enumerate(owoce, start=1):
    print(f"{i}. {owoc}")

# zip() – iterowanie po dwóch listach naraz
ceny = [3.5, 2.0, 4.2]
for owoc, cena in zip(owoce, ceny):
    print(f"{owoc}: {cena} zł")
```

🔂 Pętla `while` i słówka kluczowe `break` / `continue`

```
licznik = 0
while licznik < 5:
    print(f"Tik {licznik}")
    licznik += 1

# break – przerywa pętlę całkowicie
for liczba in range(100):
    if liczba == 10:
        break
    print(liczba)

# continue – pomija bieżącą iterację
for liczba in range(10):
    if liczba % 2 != 0:
        continue      # pomiń liczby nieparzyste
    print(liczba)      # 0, 2, 4, 6, 8
```

⚠️ **Pułapka:** `while True:` bez warunku stopu = nieskończona pętla. Zawsze miej plan

📋 Listy (`list`) — Twoja codzienna struktura danych

```
liczby = [4, 2, 7, 1, 9]

liczby.append(10)          # dodaj na koniec
liczby.insert(0, 100)      # wstaw na pozycję 0
liczby.remove(2)           # usuń pierwsze wystąpienie wartości 2
liczby.pop()                # usuń i zwróć ostatni element
liczby.sort()                # sortuj w miejscu (in-place)
liczby.sort(reverse=True)  # malejąco
sorted(liczby)                # zwraca NOWĄ posortowaną listę (nie modyfikuje)

len(liczby)                    # długość
liczby[1:3]                    # slicing – elementy od 1 do 2
liczby[-1]                      # ostatni element
```

🎭 Listy — operacje, które zaskakują początkujących

```
a = [1, 2, 3]
b = a              # ⚠ b to TA SAMA lista (referencja), nie kopia!
b.append(4)
print(a)           # [1, 2, 3, 4]  <- a też się zmieniło!

c = a.copy()       # ✅ prawdziwa kopia (płytka)
# albo: c = a[:]
# albo: import copy; c = copy.deepcopy(a)  # dla list zagnieżdżonych

# Łączenie list
x = [1, 2] + [3, 4]       # [1, 2, 3, 4]
y = [0] * 5                 # [0, 0, 0, 0, 0]

# Rozpakowywanie (unpacking)
pierwszy, *reszta = [1, 2, 3, 4]   # pierwszy=1, reszta=[2, 3, 4]
```

✨ List comprehension — "pythonowy" sposób na tworzenie list

```python
liczby = range(10)

# Styl "nie-pythonowy" (działa, ale...)
kwadraty = []
for n in liczby:
    kwadraty.append(n ** 2)

# Styl pythonowy — list comprehension 🔥
kwadraty = [n ** 2 for n in liczby]

# Z warunkiem
parzyste_kwadraty = [n ** 2 for n in liczby if n % 2 == 0]

# Zagnieżdżone (np. spłaszczanie macierzy)
macierz = [[1, 2], [3, 4], [5, 6]]
plaska = [x for wiersz in macierz for x in wiersz]   # [1, 2, 3, 4, 5, 6]
```

> 🎯 **Zasada kciuka:** jeśli comprehension nie mieści się czytelnie w jednej linii — wróć do zwykłej pętli `for`.

🔗 Krotki (`tuple`) i zbiory (`set`)

```
# Krotka – jak lista, ale NIEMUTOWALNA (immutable)
punkt = (10, 20)
x, y = punkt           # unpacking
# punkt[0] = 5         # ❌ TypeError!

# Zbiór – unikalne elementy, brak kolejności, szybkie sprawdzanie "in"
unikalne_miasta = {"Warszawa", "Kraków", "Warszawa", "Gdańsk"}
print(unikalne_miasta)    # {'Warszawa', 'Kraków', 'Gdańsk'}

a = {1, 2, 3}
b = {2, 3, 4}
a | b     # suma: {1, 2, 3, 4}
a & b     # część wspólna: {2, 3}
a - b     # różnica: {1}
```

💡 Kiedy używać czego: **lista** (kolejność + duplikaty), **krotka** (dane, które nie powinny się zmienić), **zbiór** (unikalność + szybkie `in`).

🗂️ Słowniki (`dict`) — pary klucz-wartość

```
student = {
    "imie": "Ada",
    "wiek": 29,
    "kierunki": ["Informatyka", "Data Science"],
}

student["email"] = "ada@uczelnia.pl"    # dodanie/aktualizacja
student.get("telefon", "brak danych")     # bezpieczny dostęp z domyślną wartością
"wiek" in student                          # sprawdzenie klucza → True
student.pop("wiek")                        # usuń i zwróć wartość

for klucz, wartosc in student.items():
    print(f"{klucz}: {wartosc}")

student.keys()     # wszystkie klucze
student.values()   # wszystkie wartości
```

🗂️➕ Dict comprehension i przydatne triki

```
# Dict comprehension
kwadraty = {n: n ** 2 for n in range(5)}   # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Zamiana dwóch list w słownik
klucze = ["a", "b", "c"]
wartosci = [1, 2, 3]
slownik = dict(zip(klucze, wartosci))        # {'a': 1, 'b': 2, 'c': 3}

# Zliczanie wystąpień — Counter (zamiast ręcznej pętli!)
from collections import Counter
oceny = ["A", "B", "A", "C", "A", "B"]
Counter(oceny)   # Counter({'A': 3, 'B': 2, 'C': 1})

# defaultdict — słownik z domyślną wartością (bez KeyError)
from collections import defaultdict
grupy = defaultdict(list)
grupy["info"].append("Ada")   # nie trzeba sprawdzać, czy klucz istnieje!
```

🧩 Funkcje

```python
def oblicz_bmi(waga: float, wzrost: float) -> float:
    """Oblicza BMI na podstawie wagi (kg) i wzrostu (m)."""
    return waga / (wzrost ** 2)

def przywitaj(imie: str, entuzjastycznie: bool = False) -> str:
    if entuzjastycznie:
        return f"CZEŚĆ {imie.upper()}!!! 🎉"
    return f"Cześć, {imie}."

# Argumenty pozycyjne, nazwane, domyślne
przywitaj("Ada")
przywitaj(imie="Ada", entuzjastycznie=True)

# *args i **kwargs — dowolna liczba argumentów
def suma(*liczby: float) -> float:
    return sum(liczby)

suma(1, 2, 3, 4)   # 10
```

🧠 Lambda, `map`, `filter` — funkcyjne podejście

```
kwadrat = lambda x: x ** 2
print(kwadrat(5))    # 25

liczby = [1, 2, 3, 4, 5, 6]

# map – zastosuj funkcję do każdego elementu
podwojone = list(map(lambda x: x * 2, liczby))

# filter – zostaw tylko elementy spełniające warunek
parzyste = list(filter(lambda x: x % 2 == 0, liczby))

# W praktyce: comprehension jest często czytelniejsze niż map/filter
podwojone = [x * 2 for x in liczby]
parzyste = [x for x in liczby if x % 2 == 0]
```

> 💡 PySpark bardzo często używa właśnie `lambda` wewnątrz `.map()`, `.filter()` — ta składnia wróci za chwilę!

🚨 Obsługa błędów — `try` / `except`

```
def podziel(a: float, b: float) -> float:
    try:
        wynik = a / b
    except ZeroDivisionError:
        print("⚠ Nie można dzielić przez zero!")
        return float("nan")
    except TypeError as e:
        print(f"⚠ Zły typ danych: {e}")
        return float("nan")
    else:
        print("Dzielenie zakończone sukcesem ✅")
        return wynik
    finally:
        print("Ta linia wykona się ZAWSZE")

podziel(10, 0)
podziel(10, 2)
```

🏗️ Odrobina OOP — klasy w Pythonie

```python
class Student:
    def __init__(self, imie: str, oceny: list[float]) -> None:
        self.imie = imie
        self.oceny = oceny

    def srednia(self) -> float:
        return sum(self.oceny) / len(self.oceny)

    def __repr__(self) -> str:
        return f"Student({self.imie}, średnia={self.srednia():.2f})"

ada = Student("Ada", [5.0, 4.5, 5.0])
print(ada.srednia())    # 4.83
print(ada)               # Student(Ada, średnia=4.83)
```

💡 `@dataclass` to nowoczesny skrót na klasy "trzymające dane":

```python
from dataclasses import dataclass
```

🛑 Checkpoint — co już umiemy?

- ✅ Zmienne, typy, operatory
- ✅ `if` / `elif` / `else`, `match` / `case`
- ✅ Pętle `for` / `while`, `break` / `continue`
- ✅ Listy, krotki, zbiory, słowniki + comprehensions
- ✅ Funkcje, `lambda`, `try` / `except`, odrobina OOP

To jest "czysty Python". Teraz zobaczmy, jak te same koncepty wyglądają, gdy pracujemy z **tabelami danych** 📊

🐼 pandas — "Excel na sterydach"

```
import pandas as pd

dane = {
    "imie": ["Ada", "Alan", "Grace", "Linus"],
    "wiek": [29, 41, 37, 33],
    "miasto": ["Warszawa", "Kraków", "Warszawa", "Gdańsk"],
}
df = pd.DataFrame(dane)

df.head()           # pierwsze 5 wierszy
df.shape             # (4, 3) — (wiersze, kolumny)
df.dtypes             # typy kolumn
df.info()              # podsumowanie
df.describe()           # statystyki opisowe dla kolumn numerycznych
```

> 🎯 **Analogia:** `DataFrame` to arkusz Excela, który umie liczyć miliony wierszy w ułamku sekundy i ma pełną moc Pythona za plecami.

🔍 pandas — wybieranie i filtrowanie danych

```
df["imie"]                      # pojedyncza kolumna → Series
df[["imie", "wiek"]]            # kilka kolumn → DataFrame

df.loc[0]                        # wiersz po ETYKIECIE indeksu
df.iloc[0]                       # wiersz po POZYCJI (zawsze liczbowo)

df[df["wiek"] > 35]                # filtrowanie — "maskowanie" boolowskie
df[(df["wiek"] > 30) & (df["miasto"] == "Warszawa")]   # wiele warunków (& / |, nie and/or!)

df.sort_values("wiek", ascending=False)   # sortowanie
df.query("wiek > 30")                       # alternatywna, czytelna składnia filtrowania
```

⚠️ **Pułapka:** w pandas używamy `&`, `|`, `~` zamiast `and`, `or`, `not` przy filtrowaniu!

🛠️ pandas — nowe kolumny i `apply`

```
df["pelnoletni"] = df["wiek"] >= 18                          # nowa kolumna z warunku
df["inicjaly"] = df["imie"].str[0]                             # operacje na stringach kolumnowo
df["kategoria_wieku"] = df["wiek"].apply(
    lambda w: "młody" if w < 35 else "dojrzały"
)

# Zamiast apply + lambda, pandas ma wbudowane, szybsze metody:
df["imie_wielkie"] = df["imie"].str.upper()
df["wiek_za_rok"] = df["wiek"] + 1

df.rename(columns={"imie": "name"})      # zmiana nazw kolumn
df.drop(columns=["miasto"])                 # usunięcie kolumny
```

> ⚡ **Wydajność**: operacje wektorowe (`df["wiek"] + 1`) są dużo szybsze niż `.apply()` z pętlą pod spodem — unikaj `apply` , gdy jest wbudowana alternatywa!

📊 pandas — `groupby` i agregacje

```
df.groupby("miasto")["wiek"].mean()              # średni wiek per miasto

df.groupby("miasto").agg(
    sredni_wiek=("wiek", "mean"),
    liczba_osob=("imie", "count"),
    max_wiek=("wiek", "max"),
)

# Tabela przestawna (pivot table) – jak w Excelu!
df.pivot_table(index="miasto", values="wiek", aggfunc=["mean", "count"])

df.value_counts("miasto")          # ile osób w każdym mieście
```

🎯 **Think about it** 🤔: `groupby` to dokładnie to samo, co `GROUP BY` w SQL — i zobaczymy za chwilę, że Spark SQL działa identycznie!

🔗 pandas — łączenie danych (`merge`)

```
miasta_info = pd.DataFrame({
    "miasto": ["Warszawa", "Kraków", "Gdańsk"],
    "wojewodztwo": ["mazowieckie", "małopolskie", "pomorskie"],
})

df_pelny = df.merge(miasta_info, on="miasto", how="left")
# how: "inner" | "left" | "right" | "outer" — dokładnie jak JOIN w SQL!

pd.concat([df, df], axis=0)    # sklejanie wierszy (np. kilka plików CSV)
```

`how`	Działanie
`inner`	tylko wiersze pasujące w obu tabelach
`left`	wszystkie wiersze z lewej + dopasowania z prawej
`outer`	wszystkie wiersze z obu tabel

⚠️ pandas — ograniczenia, które prowadzą nas do Sparka

- pandas trzyma **cały DataFrame w pamięci RAM** jednego komputera
- Plik 50 GB? 200 GB? Dane rozsiane po 100 serwerach? **pandas się nie nada.**
- pandas jest **jednowątkowy** z natury (poza niektórymi operacjami) — nie wykorzysta klastra maszyn

```
# To miłe i proste... dopóki dane mieszczą się w RAM
df = pd.read_csv("dane_50GB.csv")   # 💥 MemoryError
```

> 🚀 Właśnie dlatego istnieje **Apache Spark** — framework do przetwarzania danych **rozproszonego** na wielu maszynach, z PySpark jako interfejsem Pythona.

⚡ PySpark — pierwsze kroki

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("Wyklad0")
    .getOrCreate()
)

# DataFrame Sparka — koncepcyjnie BARDZO podobny do pandas!
dane = [("Ada", 29, "Warszawa"), ("Alan", 41, "Kraków")]
df_spark = spark.createDataFrame(dane, ["imie", "wiek", "miasto"])

df_spark.show()            # podgląd danych (jak df.head() w pandas)
df_spark.printSchema()      # typy kolumn
```

💡 Ten sam kod (prawie) uruchomi się zarówno na Twoim laptopie, jak i na klastrze 1000 maszyn w chmurze.

🧮 PySpark — transformacje i akcje

Spark dzieli operacje na dwie grupy — to kluczowa różnica względem pandas:

Typ	Co robi	Przykłady	Kiedy się wykonuje?
Transformacja	Opisuje PLAN obliczeń	`.filter()`, `.select()`, `.groupBy()`	**Leniwie** (lazy) — nic się nie dzieje od razu!
Akcja	Faktycznie uruchamia obliczenia	`.show()`, `.count()`, `.collect()`	**Natychmiast**

```
wynik = (
    df_spark
    .filter(df_spark.wiek > 30)     # transformacja – jeszcze nic się nie liczy
    .select("imie", "wiek")           # transformacja
)
```

🗣️ PySpark — Spark SQL (ten sam `groupby`, nowy język)

```
df_spark.createOrReplaceTempView("ludzie")

wynik = spark.sql("""
    SELECT miasto, AVG(wiek) AS sredni_wiek, COUNT(*) AS liczba
    FROM ludzie
    GROUP BY miasto
    ORDER BY sredni_wiek DESC
""")
wynik.show()

# To samo w stylu DataFrame API (identyczny wynik!)
df_spark.groupBy("miasto").agg({"wiek": "avg"})
```

💡 Spark SQL i DataFrame API to **dwa języki tego samego silnika** — wybierz ten, który Ci wygodniejszy, wydajność jest identyczna (ten sam optymalizator Catalyst).

🔁 pandas vs PySpark — ściąga porównawcza

Operacja	pandas	PySpark
Wczytanie CSV	`pd.read_csv(...)`	`spark.read.csv(...)`
Podgląd danych	`df.head()`	`df.show(5)`
Filtrowanie	`df[df.wiek > 30]`	`df.filter(df.wiek > 30)`
Nowa kolumna	`df["x"] = ...`	`df.withColumn("x", ...)`
Grupowanie	`df.groupby(...)`	`df.groupBy(...)`
Rozmiar danych	RAM jednej maszyny	Klaster (TB / PB)
Wykonanie	Natychmiastowe (eager)	Leniwe (lazy)

> 🏆 **Dobra wiadomość:** skoro znasz już pandas, PySpark DataFrame API będzie wyglądał znajomo przez 80% czasu!

⚠️ Częste pułapki — czego unikać

- ❌ Mylenie `and` / `or` z `&` / `|` w filtrowaniu pandas/Spark
- ❌ Modyfikowanie listy, podczas gdy się po niej iteruje
- ❌ Zakładanie, że `b = a` tworzy kopię listy/słownika (to tylko referencja!)
- ❌ Nadużywanie `.apply()` w pandas zamiast operacji wektorowych
- ❌ Zapominanie, że transformacje Sparka są leniwe — `.filter()` bez `.show()` / `.count()` **nic nie liczy**
- ❌ `except:` bez podania typu wyjątku — łapie też błędy, których się nie spodziewałeś

✅ Podsumowanie — kluczowe wnioski

1. **Python** to czytelny, wszechstronny język — fundament całego kursu
2. **Listy, słowniki, zbiory, krotki** — każda struktura ma swoje zastosowanie
3. **Comprehensions** (list/dict) to "pythonowy" sposób transformacji danych
4. **pandas** = tabele danych w pamięci jednej maszyny, składnia podobna do SQL/Excela
5. **PySpark** = te same koncepcje (DataFrame, `groupby`, SQL), ale **rozproszone** na klastrze
6. Transformacje Sparka są **leniwe** — dopiero akcja uruchamia obliczenia

🔮 Co dalej?

Na **Wykładzie 1** zaczynamy od **statystyki opisowej i EDA** — użyjemy dokładnie tego, co dziś poznaliśmy (pandas, pętle, funkcje), żeby zrozumieć prawdziwe dane.

🔳 Sprawdź też **Ćwiczenie 0**, gdzie przećwiczysz wszystko z tego wykładu na konkretnych zadaniach.

❓ Pytania?

Dziękuję za uwagę! 🎉

Zadanie na już: uruchom w Google Colab komórkę z `SparkSession.builder.getOrCreate()` i sprawdź, czy działa 🚀